OS Lab exercises on Multi-threaded Programs using Pthread API and Mutex/Lock for Critical section code

Q1. In this assignment, "main" thread should dynamically create 2 new threads

- One thread should loop N times printing dot character '.' each time
- Other thread should loop N times printing bar character '|' each time
- Threads are in competition for access to the screen
- Order in which characters are displayed may reflect this:
- Output may have transitions '….||||||…..|||||……||||||||||||||….'
- This lets you 'see' the threads getting and using their timeslices on the CPU
- The "main" thread should wait for completion of these two threads

(optional) Now try to change the priorities of execution of thread such that one thread finishes the printing to a large extent, if not complete, before passing the control to next thread. Show the output now with changed priorities.
Note: If your system have only a single CPU core (like most online compilers), you may not get the overlap in the pattern. However, you should still be able to run second thread before the first with changed priority value. On a multi-core system though, you should be able to see different patterns which depend on the CPU allocation of threads

Q2. Write a program using C and pthreads to perform a parallel matrix multiplication routine. The goal is to multiply an MxN matrix called A by an NxP matrix called B and then store the result into the MxP matrix called C. You can design your program in such a manner that each thread does an equal share of the work or you can have the threads run in a loop that computes single rows of the result, C. Also print time taken by each thread and compare the time taken by c program with and without pthreads. Use *gettimeofday()* to measure the time.

Note: Again if your system have only a single CPU core (like most online compilers), you may not get much difference in the time taken by parallel program and sequential program. However, on multi-core systems you should see faster execution of parallel program for sufficiently large values of M,N and P.

Q3. Calculate the addition of all the elements in a single array using multiple threads. Use mutex to implement critical section so that no two threads simultaneously update the global result variable (that stores the final addition result).

Q4. (optional) Merge sort is an algorithm where the array of numbers is split into two halves, each half is sorted recursively using merge sort, and the sorted halves are merged. For this exercise, you will use the POSIX thread libraries. You will spawn a thread to run every recursive call of merge sort i.e., at each stage you will create two threads to merge sort the two halves. When the two threads finish sorting, the parent thread merges the two halves by itself. You will also measure a couple of important parameters of this program. In particular, you will measure the time taken to sort the entire array, and the number of threads that were used to do so. You must vary the size of the array to obtain the values for time taken and number of threads. You should be able to find out the optimal number of threads that the system can support efficiently. Use *gettimeofday()* to measure the time.